

**UNIVERSIDAD NACIONAL AGRARIA DE LA SELVA
FACULTAD DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS**

**DISEÑO E IMPLEMENTACIÓN DEL MÓDULO DE
AUTENTICACIÓN Y GESTIÓN DE USUARIOS BAJO
ARQUITECTURA LIMPIA PARA EL SISTEMA DE
GESTIÓN DE INVESTIGACIÓN DE LA FIIS (SGI-FIIS)**

Documento de Especificación Técnica e Implementación

Autor:

Equipo SGI-FIIS

Curso/Proyecto:

Construcción de software II

Tingo María, Perú

2026

Resumen

El presente documento describe el diseño, la arquitectura y el proceso de implementación del módulo de Autenticación y Gestión de Usuarios para el Sistema de Gestión de Investigación de la Facultad de Ingeniería en Informática y Sistemas (SGI-FIIS). Este módulo constituye la base de seguridad y autorización del sistema, posibilitando el control de acceso de todos los actores involucrados en la investigación de la facultad (estudiantes, docentes, coordinadores, directores, decanos y administradores). La implementación técnica se ha desarrollado utilizando la plataforma Java con el framework Spring Boot, adoptando un diseño arquitectónico basado en la Arquitectura Limpia (Clean Architecture) para separar rigurosamente las reglas de negocio de la infraestructura tecnológica. Asimismo, el módulo integra autenticación local por medio de JSON Web Tokens (JWT) y un mecanismo seguro de auto-registro con verificación por código enviado a la bandeja de correo institucional, garantizando un registro controlado y un inicio de sesión seguro. La persistencia de datos se realiza en un motor PostgreSQL mediante Spring Data JPA, utilizando migraciones versionadas con la herramienta Flyway.

Palabras clave: Autenticación, Gestión de Usuarios, JWT, Auto-registro, Clean Architecture, Spring Boot, PostgreSQL, Flyway.

1. Introducción

El desarrollo científico y la gestión académica en las facultades de ingeniería demandan herramientas informáticas que automaticen los procesos de registro, evaluación y seguimiento de proyectos, tesis y publicaciones. El Sistema de Gestión de Investigación de la Facultad de Ingeniería en Informática y Sistemas (SGI-FIIS) ha sido concebido para cubrir esta necesidad. Sin embargo, para que un sistema administrativo de esta envergadura funcione adecuadamente, es imperativo contar con un sólido sistema de control de acceso y seguridad.

El módulo de Autenticación y Gestión de Usuarios representa la piedra angular de esta plataforma. Su objetivo fundamental es asegurar que cada usuario que interactúe con el sistema posea una identidad digital verificada y los permisos estrictamente necesarios para desempeñar su función. A través de este módulo se controlan las operaciones críticas de registro de investigadores, validación de credenciales institucionales, auto-registro de estudiantes mediante validación por correo electrónico institucional y el control de accesos basados en roles.

1.1. Objetivos del Módulo

El desarrollo del módulo persigue dos objetivos centrales. Por un lado, busca automatizar la administración del ciclo de vida de los usuarios dentro del sistema, permitiendo su registro, edición, desactivación temporal y el reinicio de credenciales. Por otro lado, busca proveer un mecanismo seguro y controlado de auto-registro por correo para la comunidad estudiantil, permitiéndoles crear sus propias cuentas de manera autónoma y segura.

1.2. Justificación de la Solución

La elección de integrar tanto el registro por administración centralizada como el auto-registro guiado por verificación de correo institucional responde a la diversidad de usuarios finales del sistema. Mientras que el personal administrativo y los docentes de investigación se gestionan administrativamente, los estudiantes y tesis pueden auto-registrarse de manera ágil utilizando sus cuentas institucionales verificadas por código en un proceso en dos pasos.

2. Arquitectura del Sistema (Clean Architecture)

Para garantizar la mantenibilidad, escalabilidad y testabilidad del sistema, se adoptó el patrón de diseño arquitectónico de Arquitectura Limpia (Clean Architecture). Este enfoque promueve un desacoplamiento estricto de las reglas de negocio respecto a los frameworks, bases de datos y detalles de infraestructura. La regla fundamental de este diseño establece que las dependencias deben apuntar únicamente hacia adentro, es decir, el núcleo del dominio no debe conocer ningún detalle de las capas externas.

2.1. Estructura del Proyecto y Capas

El backend del SGI-FIIS se divide en cuatro capas claramente delimitadas, estructuradas de la siguiente manera dentro del paquete de cada módulo:

1. **Capa de Dominio (Domain):** Es el núcleo del sistema. Contiene los modelos de negocio puros, los objetos de valor, los eventos del dominio y los puertos (interfaces) que definen cómo el dominio interactúa con el mundo exterior. Esta capa es completamente independiente de Spring Boot, JPA, bases de datos o cualquier librería de terceros.
2. **Capa de Aplicación (Application):** Contiene la lógica específica de la aplicación y la orquestación de los casos de uso. En ella se implementan los servicios que ejecutan flujos de negocio específicos, manipulando las entidades de dominio y comunicándose con las interfaces definidas en la capa de dominio.
3. **Capa de Infraestructura (Infrastructure):** Contiene la implementación de los detalles tecnológicos. Incluye la persistencia física (entidades JPA, repositorios Spring Data, adaptadores de bases de datos), la seguridad web (configuración de Spring Security, filtros JWT, handlers OAuth2) y la integración de servicios de terceros.
4. **Capa de Presentación (Presentation):** Actúa como la interfaz de comunicación con los clientes externos. Contiene los controladores REST encargados de recibir las peticiones HTTP, validar sintácticamente los datos de entrada, invocar los casos de uso correspondientes y retornar las respuestas adecuadas por medio de objetos de transferencia de datos (DTO).

El desacoplamiento de capas permite realizar pruebas unitarias sobre los casos de uso del sistema sin necesidad de levantar bases de datos reales o servidores web. La persistencia se inyecta en los casos de uso mediante la inversión de dependencias: la aplicación consume las interfaces (puertos) del dominio, y la infraestructura implementa dichas interfaces (adaptadores).

Esta organización y el flujo de dependencias unidireccional hacia el núcleo del dominio se representa de forma visual en la Figura 1.

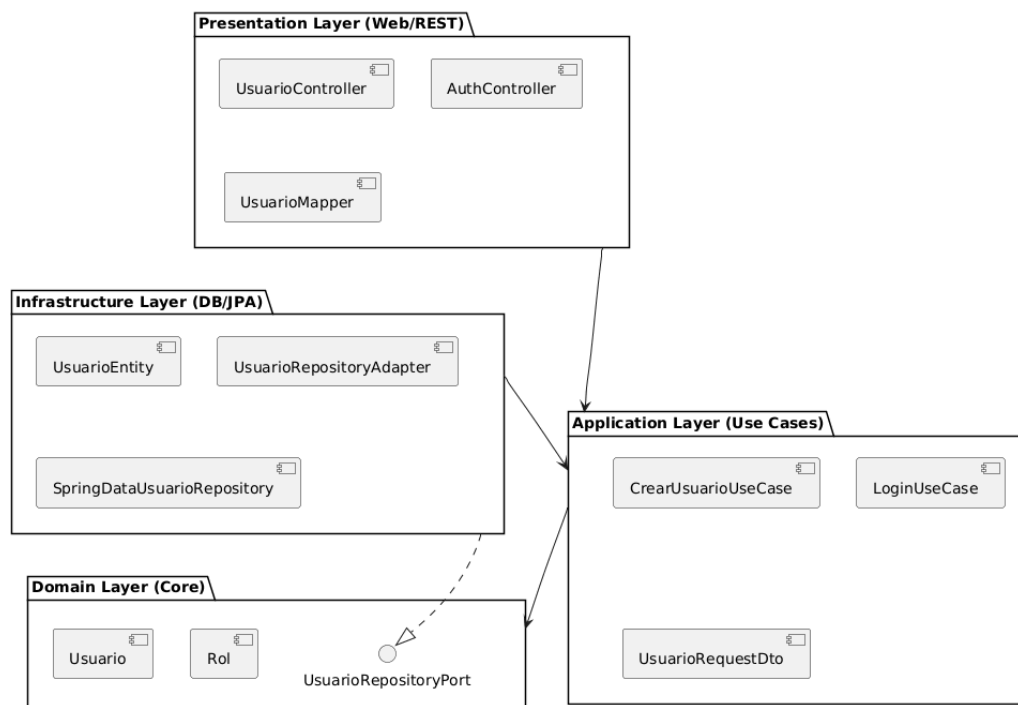


Figura 1: Arquitectura en capas y flujo de dependencias del módulo de usuarios.

2.2. Arquitectura de Despliegue e Infraestructura

Para garantizar que la aplicación backend sea fácilmente desplegable en entornos de staging y producción, el sistema se encuentra completamente contenedorizado mediante Docker y orquestado a través de Docker Compose. La infraestructura lógica del módulo se compone de cuatro nodos fundamentales: el navegador del cliente (Frontend Web), el servicio de backend Spring Boot empaquetado, la base de datos PostgreSQL y el servidor SMTP de correo para el envío de códigos de verificación.

Esta distribución física de componentes y los flujos de comunicación a través de HTTPS/REST, JDBC y protocolos de autenticación se detalla en la Figura 2.

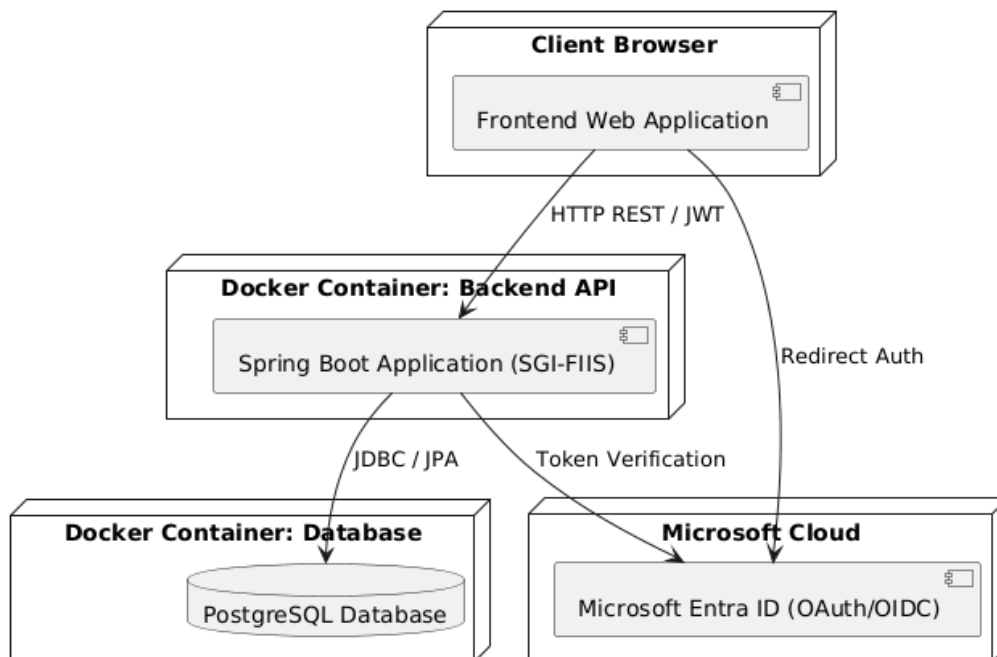


Figura 2: Diagrama de arquitectura de despliegue y flujo de integración física del sistema.

3. Modelo de Datos y Persistencia

La base de datos relacional PostgreSQL sirve como motor de almacenamiento primario del backend de SGI-FIIS. Para estructurar y poblar el esquema del módulo de seguridad de forma consistente, se utiliza Flyway como motor de migraciones, garantizando que el diseño de las tablas se aplique idénticamente en entornos de desarrollo, pruebas y producción.

3.1. Estructura de las Tablas

La seguridad del sistema se fundamenta en un modelo de control de acceso basado en roles (RBAC - Role-Based Access Control) soportado por dos tablas principales:

- **roles:** Almacena la definición de los diferentes perfiles autorizados en el sistema. Los roles predefinidos corresponden a: ADMIN, ESTUDIANTE, DOCENTE_INVESTIGADOR, COORDINADOR_GRUPO, DIRECTOR_INVESTIGACION, DECANO y EVALUADOR.
- **usuarios:** Contiene la información de perfil y las credenciales de seguridad de los usuarios registrados. Soporta tanto el login tradicional mediante campos de contraseña hasheada como el campo `oauth_provider` reservado para auditoría o integraciones futuras.

El campo `must_change_password` es un indicador booleano que obliga al usuario a actualizar su contraseña de forma mandatoria en el primer inicio de sesión. Por otro lado, la columna

oauth_provider puede almacenar información de proveedor externo en el caso de migraciones, aunque la autenticación actual del sistema se gestiona de manera puramente local y segura.

Las relaciones de cardinalidad, llaves primarias, llaves foráneas y tipos de datos físicos del esquema de persistencia se presentan de forma gráfica en la Figura 3.

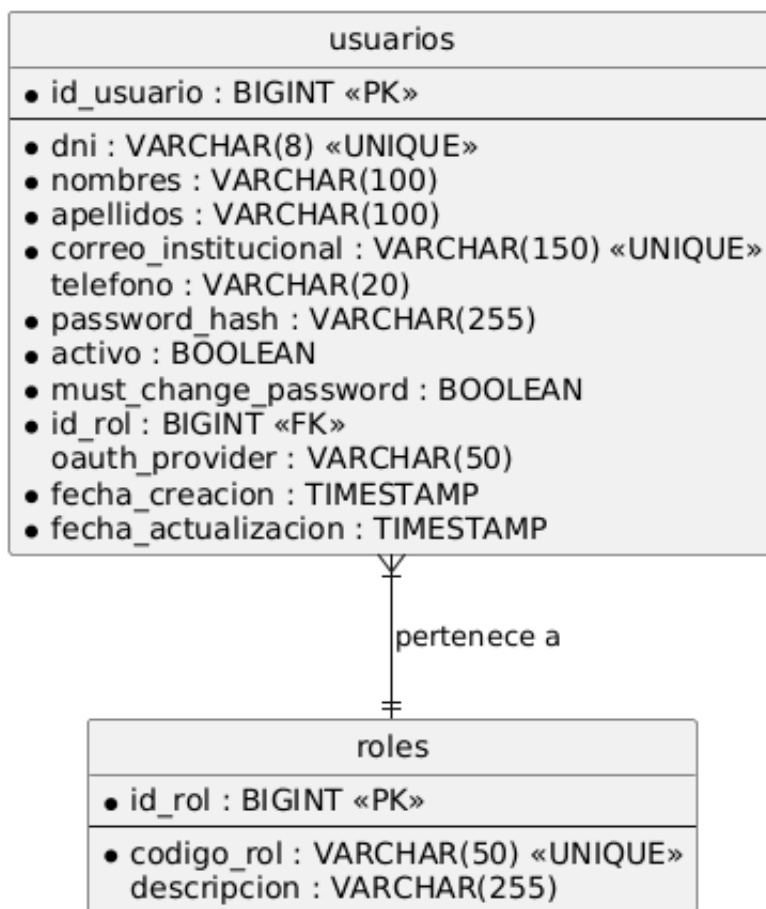


Figura 3: Diagrama Entidad-Relación (DER) de las tablas de usuarios y roles del sistema SGI-FIIS.

3.2. Script de Creación de Tablas (SQL)

La definición física de las tablas se implementa en la migración `V1__create_roles_and_usuarios.sql` y la columna OAuth se añade en `V3__add_oauth_provider_to_usuarios.sql`. A continuación, se presenta la estructura relacional expresada en código SQL:

```

1 -- Tabla de roles del sistema
2 CREATE TABLE roles (
3     id_rol          BIGSERIAL      PRIMARY KEY,
4     codigo_rol      VARCHAR(50)   NOT NULL UNIQUE,

```

```
5      descripcion  VARCHAR(255)
6  );
7
8  -- Tabla de usuarios del sistema
9  CREATE TABLE usuarios (
10     id_usuario    BIGSERIAL      PRIMARY KEY,
11     dni           VARCHAR(8)     NOT NULL UNIQUE,
12     nombres       VARCHAR(100)   NOT NULL,
13     apellidos     VARCHAR(100)   NOT NULL,
14     correo_institucional VARCHAR(150) NOT NULL UNIQUE,
15     telefono      VARCHAR(20),
16     password_hash VARCHAR(255)   NOT NULL,
17     activo        BOOLEAN        NOT NULL DEFAULT TRUE,
18     must_change_password BOOLEAN  NOT NULL DEFAULT TRUE,
19     id_rol        BIGINT         NOT NULL REFERENCES roles(
20         id_rol),
21     oauth_provider VARCHAR(50), -- null si es login local
22     fecha_creacion  TIMESTAMP    NOT NULL DEFAULT NOW(),
23     fecha_actualizacion  TIMESTAMP NOT NULL DEFAULT NOW()
24 );
25
26 -- Indices para búsquedas frecuentes
27 CREATE INDEX idx_usuarios_rol ON usuarios(id_rol);
28 CREATE INDEX idx_usuarios_activo ON usuarios(activo);
29 CREATE INDEX idx_usuarios_dni ON usuarios(dni);
30 CREATE INDEX idx_usuarios_correo ON usuarios(correo_institucional);
31 CREATE INDEX idx_usuarios_oauth_provider ON usuarios(oauth_provider);
```

Listing 1: Esquema físico de base de datos para roles y usuarios.

Las restricciones de integridad aseguran la unicidad del DNI y del correo institucional, previniendo duplicidades. Los índices se han ubicado estratégicamente en las columnas más utilizadas en las cláusulas WHERE de las consultas internas, tales como la verificación del correo durante el login y la búsqueda de usuarios activos.

4. Especificación Funcional y Flujos de Control

El comportamiento operativo del módulo abarca el inicio de sesión convencional con JWT, el auto-registro con verificación por código por correo, el ciclo de cambio de contraseñas y la administración general de cuentas de usuario por parte de los administradores.

Para modelar y estructurar el comportamiento del sistema, la interacción de los distintos actores con las funcionalidades de seguridad y administración se representa de forma segmentada y clara mediante diagramas de casos de uso UML específicos por cada uno de los siete roles del sistema. Esta organización individualizada previene el cruce de líneas y la sobrecarga visual en la documentación.

La Figura 4 ilustra las acciones de administración técnica de usuarios, líneas de investigación y membresías de grupos a cargo del Administrador.

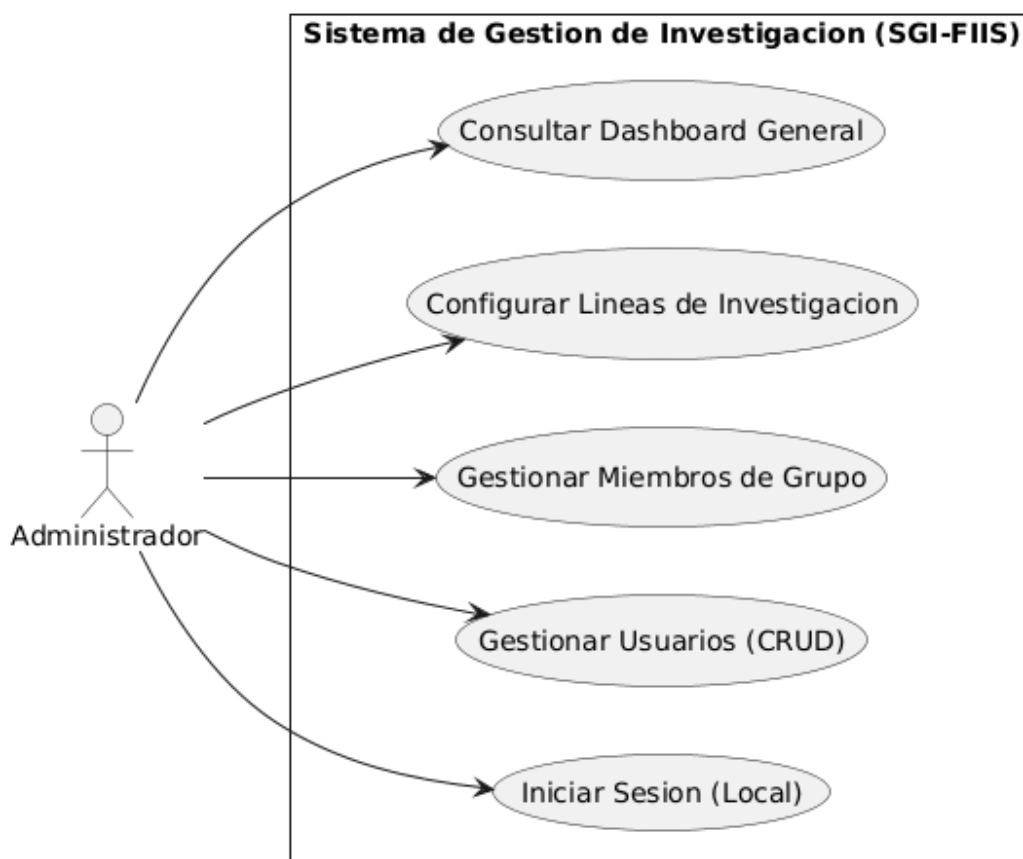


Figura 4: Diagrama de casos de uso UML para el rol de Administrador.

Por otro lado, la interacción del Estudiante o Tesista con el auto-registro, inicio de sesión local, el registro de planes de tesis y la subsanación de observaciones se representa en la Figura 5.

Asimismo, las funcionalidades específicas del Docente Investigador para registrar proyectos de

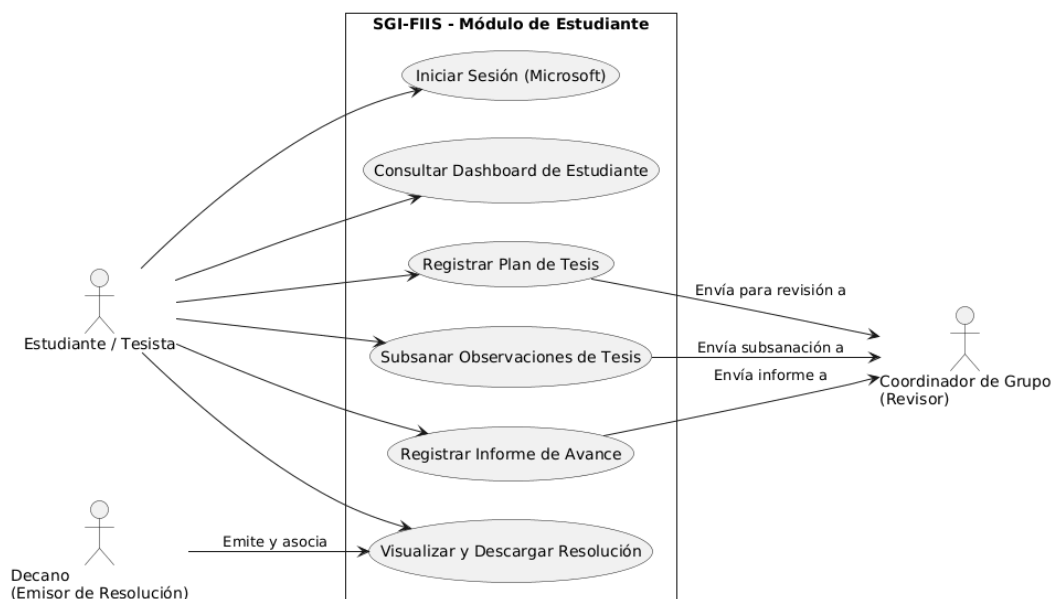


Figura 5: Diagrama de casos de uso UML para el rol de Estudiante / Tesista.

investigación y subsanar sus respectivas observaciones se muestran en la Figura 6.

El flujo de revisión y control académico por parte del Coordinador de Grupo se detalla en la Figura 7, mostrando su interacción con el Solicitante (docente o estudiante) y el Director de Investigación como siguiente revisor.

La Figura 8 describe las interacciones del Director de Investigación para evaluar trámites pre-revisados por el Coordinador, derivar aprobaciones al Decanato, devolver observaciones y registrar las convocatorias oficiales.

Por su parte, las acciones del Decano para revisar expedientes avalados por Dirección, registrar resoluciones académicas y formalizar la aprobación de proyectos se ilustran en la Figura 9.

Finalmente, las interacciones específicas del Docente Evaluador para dictaminar los proyectos de investigación asignados por Dirección y reportar sus calificaciones se especifican en la Figura 10.

4.1. Autenticación Convencional (JWT)

Cuando un usuario inicia sesión con correo y contraseña, el sistema valida que las credenciales coincidan con la firma hasheada (BCrypt) y verifica que la cuenta se encuentre activa. Si la autenticación es exitosa, se genera un token JSON Web Token (JWT) firmado digitalmente que expira tras un periodo predeterminado. El frontend debe enviar este token en el encabezado HTTP `Authorization: Bearer <token>` para cada consulta protegida subsiguiente.

El flujo operativo descrito anteriormente y las transacciones de validación de identidad corres-

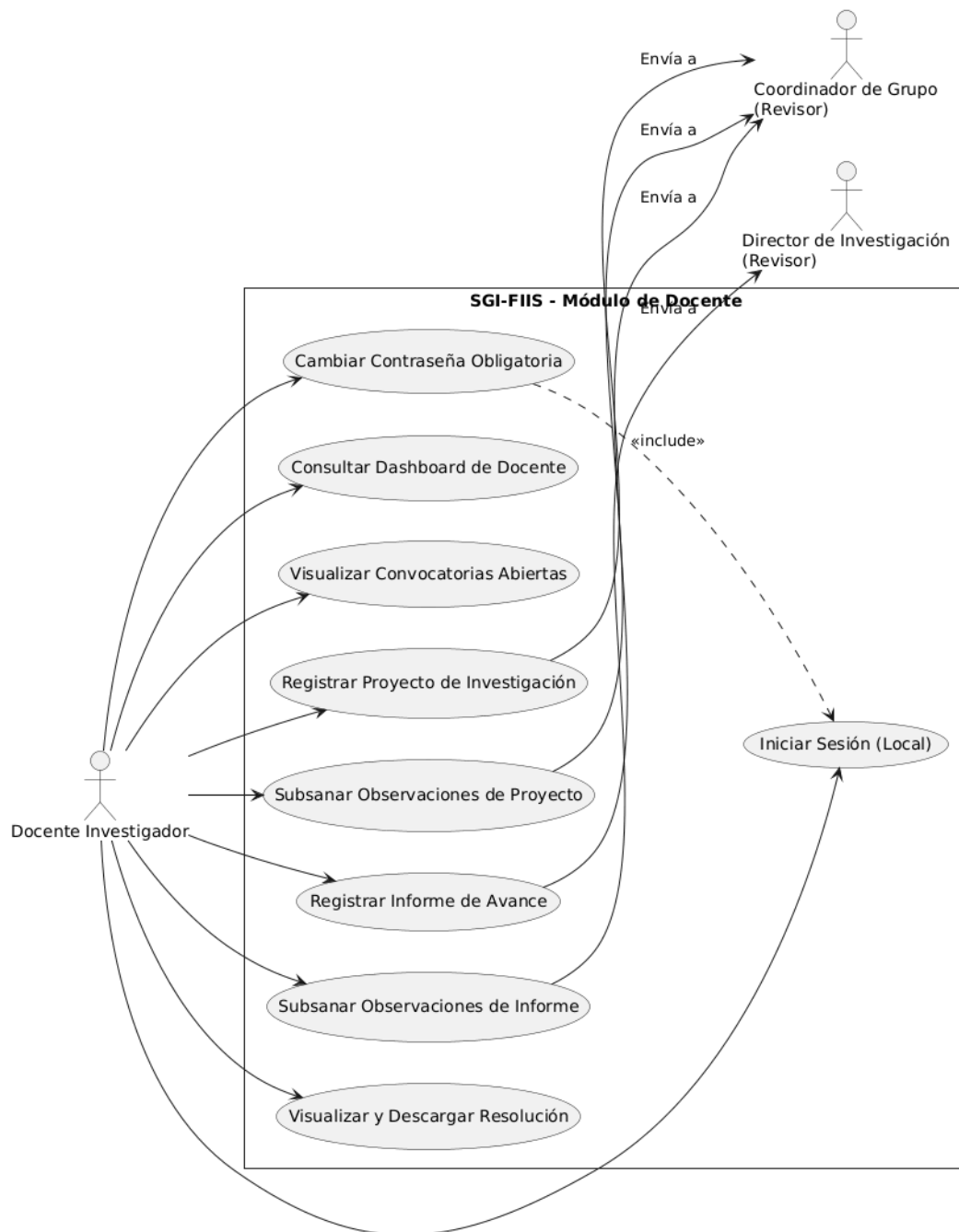


Figura 6: Diagrama de casos de uso UML para el rol de Docente Investigador.

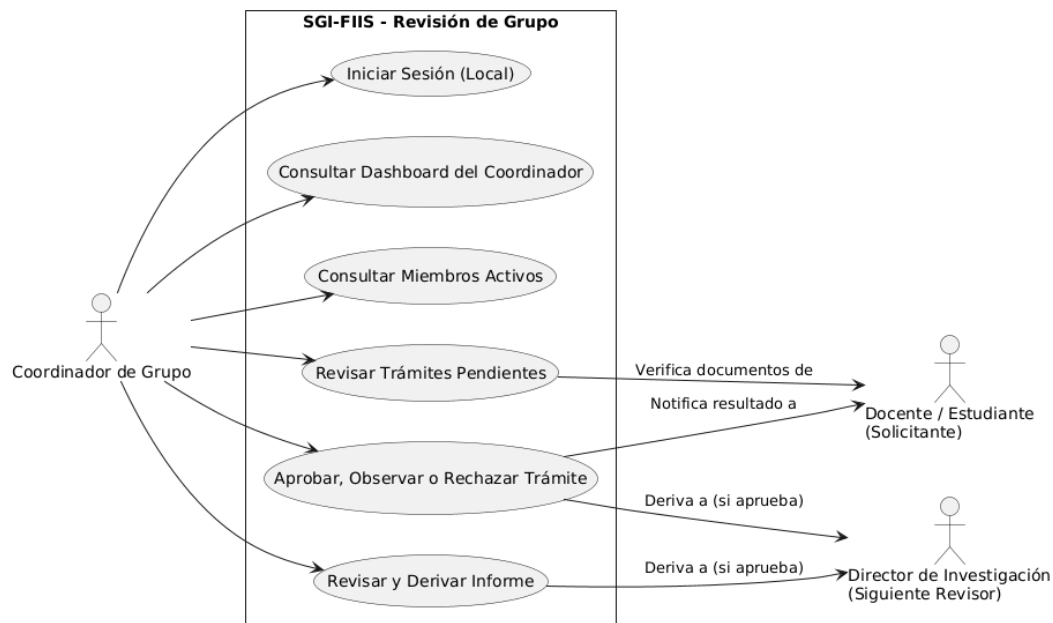


Figura 7: Diagrama de casos de uso UML para el rol de Coordinador de Grupo.

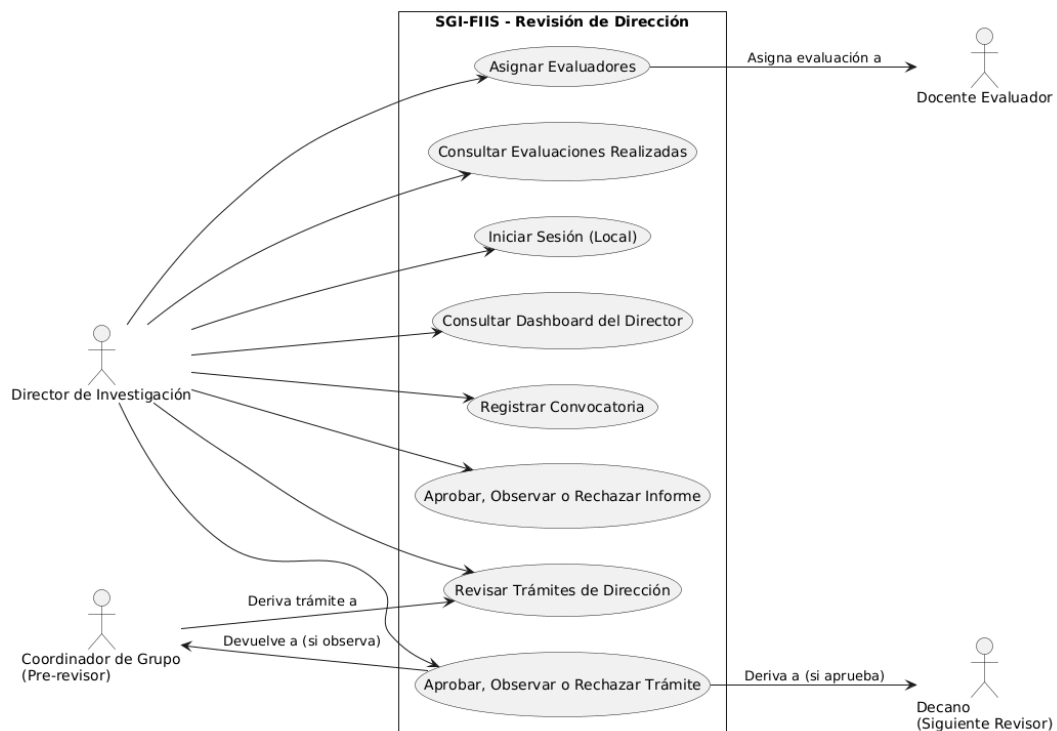


Figura 8: Diagrama de casos de uso UML para el rol de Director de Investigación.



Figura 9: Diagrama de casos de uso UML para el rol de Decano.

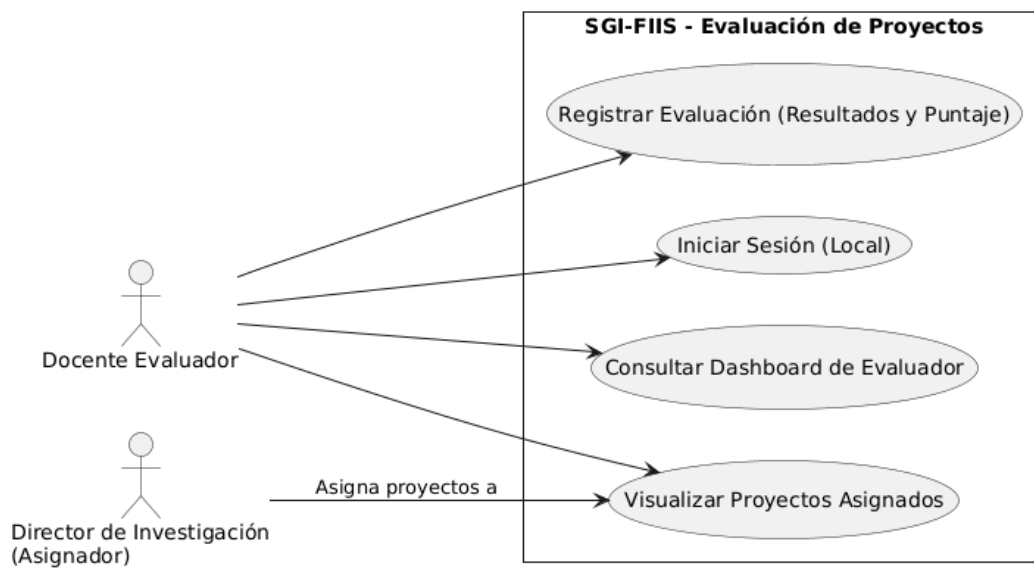


Figura 10: Diagrama de casos de uso UML para el rol de Docente Evaluador.

pendientes se visualizan gráficamente en el diagrama de secuencia de la Figura 11.

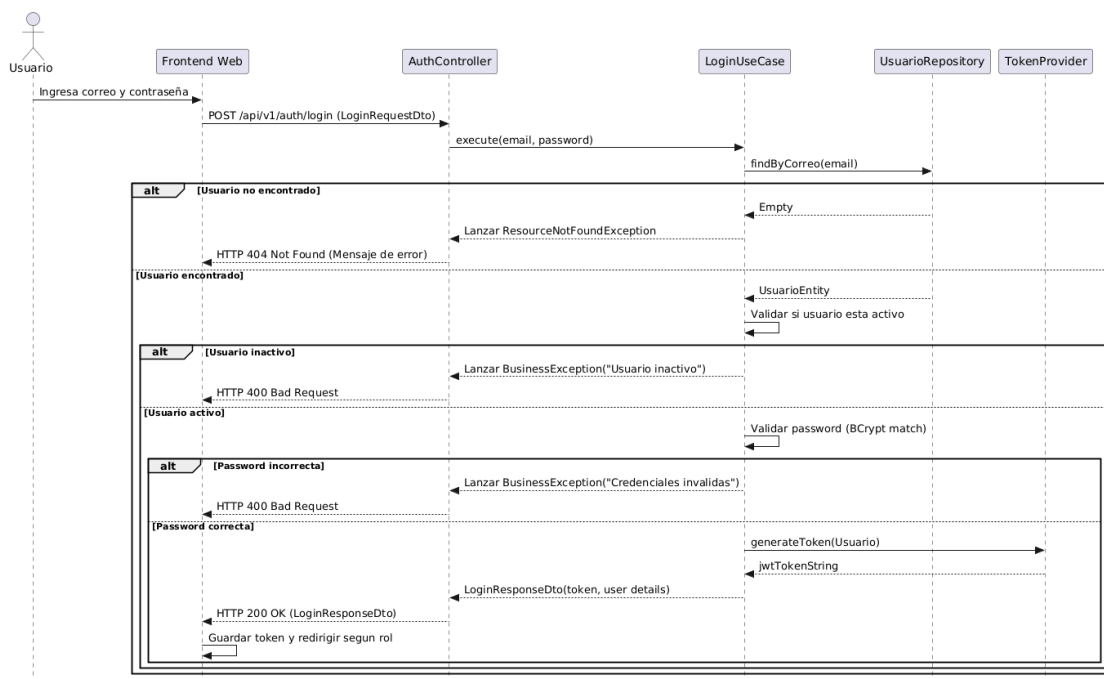


Figura 11: Diagrama de secuencia del proceso de autenticación local y generación de JWT.

4.2. Auto-registro de Estudiantes con Verificación por Correo

El auto-registro de nuevos estudiantes o tesisistas se gestiona mediante un flujo en dos pasos. El usuario envía sus datos iniciales (incluyendo su DNI, nombres, apellidos y correo institucional) al endpoint de registro. El sistema valida que el dominio del correo sea institucional (`.edu.pe`), verifica la inexistencia de duplicados en la base de datos y genera un código de verificación aleatorio de 6 dígitos que guarda temporalmente en memoria con un tiempo de expiración de 5 minutos.

En el segundo paso, el estudiante ingresa el código recibido en su correo electrónico. Si la clave temporal es válida y no ha expirado, el sistema cifra su contraseña con `BCrypt`, persiste su cuenta directamente en la base de datos física (asignándole el rol por defecto `ESTUDIANTE`), elimina los datos temporales en memoria y le provee un JWT firmado digitalmente para iniciar sesión de forma inmediata.

Este flujo de registro y verificación por código en memoria temporal se describe técnicamente en detalle en el archivo de especificación de flujos del sistema.

4.3. Flujo de Cambio de Contraseña Obligatoria

Para prevenir problemas de seguridad derivados del aprovisionamiento manual de cuentas por el administrador, el sistema exige que todos los usuarios que cuenten con inicio de sesión local

cambien su contraseña por defecto al ingresar por primera vez. Si el indicador `must_change_password` está establecido en verdadero (`true`), el filtro de seguridad y el backend restringirán el acceso a los recursos comunes y guiarán al usuario hacia el formulario de actualización de contraseña. Tras un cambio exitoso, el indicador se establece en falso (`false`), liberando el acceso regular del usuario.

El flujo que rige la obligación de actualizar las credenciales locales de primer ingreso para mitigar riesgos de seguridad y desactivar el indicador de reinicio obligatorio se representa visualmente en la Figura 12.

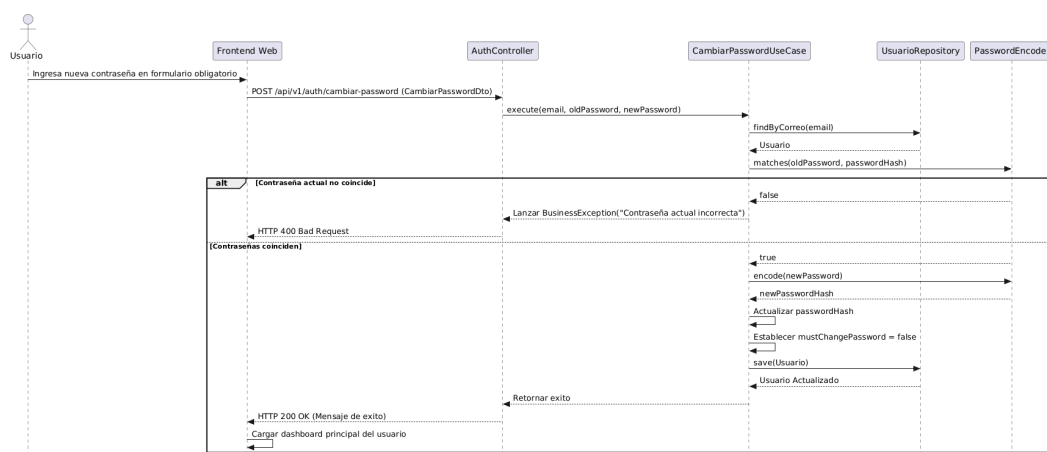


Figura 12: Diagrama de secuencia de cambio obligatorio de contraseña en el primer inicio de sesión.

4.4. Gestión de Usuarios y Creación de Cuentas

El Administrador es el actor responsable de gestionar el ciclo de vida de los usuarios dentro del sistema. Al registrar un usuario, el caso de uso realiza validaciones de reglas de negocio: valida que el DNI no esté duplicado, que el correo institucional sea único y, en caso de no ingresarse un correo manualmente, el sistema genera automáticamente la dirección bajo el formato establecido por la regla institucional: `primer_nombre.primer_apellido@unas.edu.pe`.

Este flujo interactivo y las validaciones de duplicidad se representan de forma secuencial en la Figura 13.

Además, la seguridad y el principio de mínimo privilegio en el sistema se gobiernan mediante una matriz de autorización detallada. Esta matriz define los permisos de los diferentes roles para cada uno de los endpoints de API críticos del módulo. Las políticas y privilegios correspondientes se estructuran formalmente en la Tabla 1.

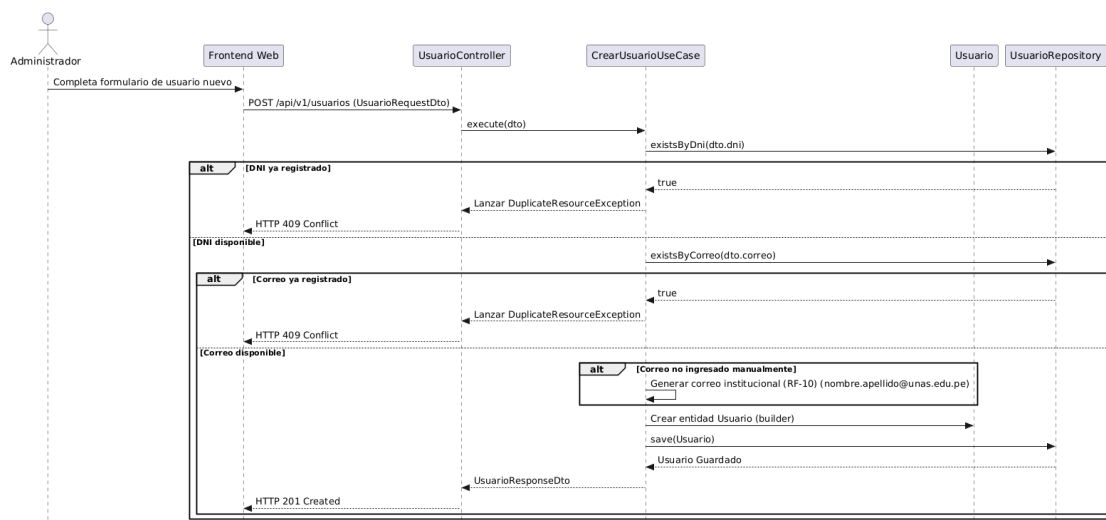


Figura 13: Diagrama de secuencia del proceso de creación y registro de usuarios por el Administrador.

API Endpoint / Recurso	ADM	EST	DOC	COOR	DIR	DEC	EVA
POST /api/v1/auth/login	Permitir	Permitir	Permitir	Permitir	Permitir	Permitir	Permitir
POST /api/v1/auth/cambiar-password	Permitir	Permitir	Permitir	Permitir	Permitir	Permitir	Permitir
POST /api/v1/usuarios	Permitir	Denegar	Denegar	Denegar	Denegar	Denegar	Denegar
GET /api/v1/usuarios	Permitir	Denegar	Denegar	Denegar	Denegar	Denegar	Denegar
GET /api/v1/usuarios/{id}	Permitir	Denegar	Denegar	Denegar	Denegar	Denegar	Denegar
PUT /api/v1/usuarios/{id}	Permitir	Denegar	Denegar	Denegar	Denegar	Denegar	Denegar
PATCH /api/v1/usuarios/{id}/estado	Permitir	Denegar	Denegar	Denegar	Denegar	Denegar	Denegar
POST /api/v1/usuarios/{id}/reset	Permitir	Denegar	Denegar	Denegar	Denegar	Denegar	Denegar
GET /api/v1/roles	Permitir	Denegar	Denegar	Denegar	Denegar	Denegar	Denegar

Cuadro 1: Matriz de control de acceso basada en roles (RBAC) para el módulo de seguridad (ADM: Administrador, EST: Estudiante, DOC: Docente Investigador, COOR: Coordinador, DIR: Director, DEC: Decano, EVA: Evaluador).

5. Detalles de Implementación Técnica

La materialización del módulo en Spring Boot se rige por la estructuración de componentes desacoplados de infraestructura que operan en conjunto con los casos de uso definidos en la capa de aplicación.

5.1. Filtro de Autenticación JWT

El archivo `JwtAuthenticationFilter.java` hereda de `OncePerRequestFilter` para interceptar cada petición HTTP que entra al servidor. Se encarga de extraer la cabecera `Authorization`, extraer el token JWT, verificar su firma con la clave secreta y poblar el contexto de seguridad de Spring Security (`SecurityContextHolder`) si el token es válido. Esto permite aplicar directivas de seguridad a nivel de rutas y controladores de forma transparente.

5.2. Configuración de Seguridad

La configuración general de seguridad se define en `SecurityConfig.java`. Este componente acopla el soporte para tokens JWT locales, obligando a que toda comunicación REST sea completamente stateless. A continuación, se presenta este código:

```
1 package com.sgi.fiis.auth.infrastructure.security;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.annotation.web.builders.
    HttpSecurity;
6 import org.springframework.security.config.annotation.web.configuration
    .EnableWebSecurity;
7 import org.springframework.security.config.http.SessionCreationPolicy;
8 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder
    ;
9 import org.springframework.security.crypto.password.PasswordEncoder;
10 import org.springframework.security.web.SecurityFilterChain;
11 import org.springframework.security.web.authentication.
    UsernamePasswordAuthenticationFilter;
12
13 @Configuration
14 @EnableWebSecurity
15 public class SecurityConfig {
16
```

```
17     private final JwtAuthenticationFilter jwtAuthFilter;
18
19     public SecurityConfig(JwtAuthenticationFilter jwtAuthFilter) {
20         this.jwtAuthFilter = jwtAuthFilter;
21     }
22
23     @Bean
24     public SecurityFilterChain securityFilterChain(HttpSecurity http) {
25         try {
26             http
27                 .csrf(csrf -> csrf.disable())
28                 .sessionManagement(session -> session.
29                     sessionCreationPolicy(SessionCreationPolicy.
30                         STATELESS))
31                 .authorizeHttpRequests(auth -> auth
32                     .requestMatchers("/api/v1/auth/**").permitAll()
33                     .requestMatchers("/health").permitAll()
34                     .requestMatchers("/api/v1/usuarios/**").hasRole("
35                         ADMIN")
36                     .requestMatchers("/api/v1/roles/**").hasRole("ADMIN
37                         ")
38                     .anyRequest().authenticated()
39                 )
40                 .addFilterBefore(jwtAuthFilter,
41                     UsernamePasswordAuthenticationFilter.class)
42                 .exceptionHandling(exceptions -> exceptions
43                     .defaultAuthenticationEntryPointFor(
44                         new org.springframework.security.web.
45                             authentication.HttpStatusEntryPoint(org.
46                                 springframework.http.HttpStatus.UNAUTHORIZED
47                             ),
48                         org.springframework.security.web.servlet.util.
49                             matcher.PathPatternRequestMatcher.
50                             pathPattern("/api/**")
51                     )
52                 );
53
54         return http.build();
55     }
```

```
45         } catch (Exception e) {  
46             throw new IllegalStateException("Failed to configure  
47                 security filter chain", e);  
48         }  
49  
50     @Bean  
51     public PasswordEncoder passwordEncoder() {  
52         return new BCryptPasswordEncoder();  
53     }  
54 }
```

Listing 2: Configuración de Spring Security para JWT.

Esta configuración establece una arquitectura sin estado (*Stateless*) para todas las peticiones HTTP regulares que llevan cabeceras de autorización de tipo Bearer, eliminando completamente las sesiones en el servidor backend y disminuyendo riesgos de inyecciones y ataques CSRF.

6. Conclusiones y Recomendaciones

La construcción e implementación del módulo de seguridad para el SGI-FIIS aporta las siguientes conclusiones:

- La adopción de la Arquitectura Limpia (Clean Architecture) reduce sustancialmente el acoplamiento del núcleo de la aplicación, facilitando la adición de futuros proveedores de identidad o cambios en el motor de base de datos sin afectar las reglas de negocio establecidas para los usuarios y los roles.
- El mecanismo de auto-registro local con verificación por código por correo institucional responde eficientemente al contexto universitario, promoviendo el aprovisionamiento automatizado y ágil de cuentas estudiantiles bajo un entorno de seguridad administrado.
- La inclusión del cambio de contraseña mandatorio y la deshabilitación lógica de cuentas en lugar de eliminaciones físicas garantizan la trazabilidad histórica de los trámites y proyectos en los cuales los usuarios participaron, mitigando riesgos de inconsistencia referencial en la base de datos del sistema de investigación.

Se recomienda implementar políticas rigurosas de expiración periódica de tokens JWT para fortalecer la integridad del sistema ante posibles fugas de credenciales locales en las terminales físicas de los usuarios de la facultad.

7. Referencias

- Beck, K. (2003). *Test-Driven Development: By Example*. Addison-Wesley Professional.
- Martin, R. C. (2018). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Spring Projects. (2026). *Spring Security Reference*. VMware Tanzu. <https://docs.spring.io/spring-security/reference/>